

Software Implementation Specification: Backend Receiver & Analytics Dashboard (V3)

Date: May 23, 2026

This document details the software architecture, data receiving protocols, edge-side optimizations, production-ready tech stack selection, and web-based dashboard analytics for the *Vehicle Excessive Noise and Vibration Detection System*. This version establishes the technical foundation required to build, deploy, and evaluate the cloud-side framework.

1. Enhanced System Architecture Overview

The software environment operates on a decoupled client-server model optimized for edge computing conditions. It is structurally divided into three core resilient layers: the Edge Layer (with localized message queue fault tolerance), the Ingestion Layer (REST API Endpoints and analytical processing modules), and the Presentation Layer (Dynamic Web Dashboard).

2. Software Tech Stack Specification

To implement this architecture effectively, a modular, asynchronous, and high-performance stack is selected. This ensures that telemetry data packets are ingested seamlessly while administrative dashboards remain instantly reactive.

Architecture Component	Technology Selected	Engineering Rationale
Backend REST API	Python (FastAPI)	Utilizes ASGI standards for blazing-fast concurrent asynchronous execution; includes automatic JSON data validation via Pydantic and generates interactive Swagger testing documentation.
Frontend Dashboard	React.js	Component-based virtual DOM engine allows the real-time alert feed to update instantly without reloading the rest of the application layouts.
UI Styling	TailwindCSS	A utility-first CSS framework enabling fast, responsive creation of professional

Architecture Component	Technology Selected	Engineering Rationale
		tables, status indicators, and metric highlight cards.
Real-Time Pipeline	WebSockets (Socket.io)	Replaces slow HTTP polling with a permanent, bi-directional pathway, instantly pushing live violation packets from backend to frontend.
Relational Database	SQLite (Proto) / PostgreSQL (Prod)	SQLite provides serverless simplicity during initial local development; PostgreSQL delivers production-grade query execution for historic traffic analytics.
Media Storage Block	Local Filesystem / Object Storage	Keeps the SQL database lightweight by saving compressed JPEGs to a file directory or storage bucket (MinIO/S3), indexing only the string URI string paths.
Data Visualization	Chart.js / Recharts	Provides dynamic canvas rendering for multi-axis line graphs and bar metrics plotting ambient noise distributions.

3. Edge-Side Fault Tolerance & Optimization

3.1 Edge-Side Data Buffering (Offline Queueing)

To prevent data loss resulting from localized Wi-Fi or cellular network dropouts, the edge computing unit must not send blocking direct HTTP requests. The following pipeline must be implemented locally:

- Upon triggering a violation event, telemetry metrics and image file paths are written immediately to a local database queue (e.g., SQLite).
- A dedicated background service continuously checks network socket connectivity to the main server dashboard.
- When a connection is verified, the service flushes the queue sequentially using an exponential backoff retry mechanism, safely clearing the local cache upon receipt of an HTTP 201 Success status.

3.2 Multi-Modal Event Deduplication (Time-Window Debouncing)

Because slow-moving vehicles emitting high-volume bass could continuously trigger the threshold checks within a single pass, a software-level debounce mechanism is mandatory to prevent server data flooding:

- **Hardware Lockout Timer:** Once a true violation logic condition is met, a non-blocking hardware cooldown timer (typically 3 to 5 seconds) is activated, preventing subsequent sensor triggers from capturing duplicate incidents for the same vehicle.
- **Server Clustering:** If multiple consecutive spikes occur within an ultra-tight temporal window, the Ingestion Layer automatically groups the matching telemetry entries into a single master incident file before archiving it to the database.

3.3 Asynchronous Uploads & Image Compression

Transmitting high-resolution photographic evidence directly within the primary telemetry network loop delays subsequent sensor capture operations. The image processing pipeline must split data types:

1. The lightweight JSON telemetry packet (decibel values, vibration levels, and text characters) is posted instantly and asynchronously to the REST API.
2. The raw image frame is compressed into an optimized JPEG format using edge image libraries (such as OpenCV or Pillow).
3. The compressed visual evidence is transferred via a background worker thread to an Object Storage bucket (e.g., AWS S3 or a local MinIO instances), returning a clean reference URL that links back to the SQL database record.

4. Ingestion Layer & API Specification

The Data Receiving Center hosts HTTP server endpoints configured to accept unified payloads while managing dynamic thresholds remotely via a configuration endpoint.

4.1 Violation Ingestion Endpoint

- **Route:** POST /api/v1/violations
- **Content-Type:** application/json

```
{
  "device_id": "EDGE_NODE_JAMMU_01",
  "timestamp": "2026-05-23T21:45:00Z",
  "metrics": {
    "audio_db": 92.4,
    "vibration_g": 1.85,
    "peak_frequency_hz": 45.2
  },
  "evidence": {
    "license_plate_text": "JK02BY8765",
    "confidence_score": 0.94,
    "image_url":
```

```
"https://storage.local/evidences/20260523_JK02BY8765.jpg"
    }
}
```

4.2 Remote Configuration API (Dynamic Thresholds)

To avoid hardcoding parameters that change based on environment rush hours, edge devices pull context variables periodically:

- **Route:** GET /api/v1/config?device_id=EDGE_NODE_JAMMU_01
- **Response Body:** {"audio_threshold_db": 85.0, "vibration_threshold_g": 1.2, "cooldown_seconds": 5}

5. Presentation Layer & Analytics Dashboard

The dashboard renders incoming telemetry in real-time while integrating intelligent processing layers to optimize law enforcement analysis workflow.

5.1 Interface Elements

- **Dynamic Live Alert Feed:** Uses WebSockets to stream incoming incidents onto a monitoring interface without page refreshes. Incidents crossing extreme bounds are highlighted with critical warning states.
- **Spatial Heatmap Matrix:** Collects historic violation counts categorized by device location and time blocks to visually depict high-incidence urban clusters.
- **OCR Confidence Gating:** If the Edge computer visions library reports an OCR character confidence rating under 80% (0.80), the UI routes the card to a dedicated "Manual Review Queue" to ensure absolute reporting integrity.

6. Reference Python Backend Script (FastAPI Core)

This script replaces the conceptual Flask snippet with an asynchronous FastAPI application incorporating typing, database persistence routing, and config maps:

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel
import sqlite3
from datetime import datetime

app = FastAPI(title="SonicSentry Backend Receiver", version="2.0.0")

class Metrics(BaseModel):
    audio_db: float
    vibration_g: float
    peak_frequency_hz: float

class Evidence(BaseModel):
    license_plate_text: str
    confidence_score: float
```

```

        image_url: str

class ViolationPayload(BaseModel):
    device_id: str
    timestamp: str
    metrics: Metrics
    evidence: Evidence

def init_db():
    conn = sqlite3.connect('sonicsentry_prod.db')
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS incidents (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            device_id TEXT, timestamp TEXT, audio_db REAL, vibration_g
REAL, license_text TEXT, image_path TEXT
        )
    ''')
    conn.commit()
    conn.close()

@app.on_event("startup")
def startup_event():
    init_db()

@app.post("/api/v1/violations", status_code=status.HTTP_201_CREATED)
async def log_violation(payload: ViolationPayload):
    try:
        conn = sqlite3.connect('sonicsentry_prod.db')
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO incidents (device_id, timestamp, audio_db,
vibration_g, license_text, image_path)
            VALUES (?, ?, ?, ?, ?, ?)
        ''', (payload.device_id, payload.timestamp,
payload.metrics.audio_db,
            payload.metrics.vibration_g,
payload.evidence.license_plate_text, payload.evidence.image_url))
        conn.commit()
        conn.close()
        # In a complete build, trigger WebSocket broadcast to React
dashboard here
        return {"status": "success", "detail": "Incident recorded
asynchronously"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Internal Storage
Failure: {str(e)}")

```

```
@app.get("/api/v1/config")
async def get_edge_configuration(device_id: str):
    return {
        "device_id": device_id,
        "audio_threshold_db": 85.0,
        "vibration_threshold_g": 1.5,
        "cooldown_seconds": 5,
        "active_frequency_range": [30, 60]
    }
```